RANKTIER: EFFICIENT MIXTURE-OF-EXPERTS PREFILL ON AHEAD-OF-TIME COMPILED INFERENCE ACCELERATORS

Anonymous Authors[1]

ABSTRACT

Mixture-of-experts (MoE) models route each token to a few experts chosen at run time, so the work each expert receives changes from input to input and from layer to layer. Server inference accelerators such as Qualcomm Cloud AI 100 run programs compiled ahead of time, with shapes, partitions and schedules fixed before the first request. Uniform expert capacities accept any routing but execute padded work, and capacities fitted to observed loads fail when the busy experts change. We present RankTier, which adapts expert work inside a static multi-device program while keeping the aggregation of expert outputs efficient. Each device ranks its experts by load at run time and binds compiled capacity tiers to load ranks, whose order statistics are bounded and stable across workloads. Expert outputs return to their tokens through a layout that keeps each device's rows on that device and each token's sum on one core. The two parts must be designed together. On Qwen3-30B-A3B across the four SoCs of a Cloud AI 100 Ultra card, ranked tiers cut full-model prefill latency at 512-token chunks by 27% with RankTier's aggregation but by only 4% with a repaired dense combine, and aggregation graphs that compute bit-identical results but express device ownership through index values or slices run 3.4–5.3× slower. RankTier prefills 128-, 256- and 512-token chunks 4.74×, 3.65× and 4.68× faster than the production compilation and 2.37×, 3.55× and 4.56× faster than an expert-parallel program with the exported combine. Its calibrated capacities overflowed in none of 1872 evaluation chunks, and a forced overflow is recovered bit-exactly.

1 INTRODUCTION

Mixture-of-experts (MoE) layers let a language model grow its parameter count without a matching growth in per-token compute: a router sends each token to a few of many experts, and only those experts run (Shazeer et al., 2017; Fedus et al., 2022). Qwen3-30B-A3B routes each token to 8 of 128 experts in each of its 48 layers (Yang et al., 2025), and Mixtral and DeepSeek-V3 follow the same pattern (Jiang et al., 2024; DeepSeek-AI, 2024). During prefill a serving system processes the prompt in chunks of T tokens (Agrawal et al., 2024), and the router distributes each chunk's $8T$ assignments unevenly over the experts, differently in every layer and for every input. Efficient execution should therefore give different experts different amounts of work.

Server inference accelerators make that hard. Qualcomm Cloud AI 100 (Chatha, 2021), like XLA-compiled TPU programs (Lepikhin et al., 2021) and the statically scheduled Groq TSP (Abts et al., 2020), runs a model as a program compiled ahead of time: tensor shapes, the partition of every operator over cores and devices, and the schedule are fixed before the first request, and the compiler rejects control flow that depends on data. Expert work must be expressed through compiled shapes. A uniform capacity of T rows per expert accepts any routing, but it executes padded rows, 16× the routed assignments for this model, and the compiler skips them only in whole row tiles. Capacities fitted to observed loads do not escape this, because which experts are busy changes with the workload (§3.1).

Reducing expert work is only half of the problem. Expert outputs must return to their tokens and be summed over each token's experts, and the shapes and stages that capacity savings introduce change how that aggregation executes. In a controlled experiment that holds everything else fixed, ranked capacities cut the full-model latency of 512-token chunks by 27% with RankTier's aggregation but by only 4% with a repaired dense combine, and at 128 tokens they make the dense version 37% slower (§3.2). Choosing capacities to minimize padded rows is an incomplete objective: capacities and aggregation have to be designed together.

Efficient aggregation in turn depends on how the graph ex-

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

presses it. The mathematics is standard: each device sums the rows of the experts it computed into per-token partials, and the partials are summed across devices, as GPU systems do with sparse kernels (Hwang et al., 2023; vLLM Project, 2026). Through a graph compiler, however, equivalent expressions of that computation execute very differently. Three graphs that produce bit-identical logits differ only in how they tell the compiler which device owns which rows: global row indices and explicit per-device slices both make it ship expert outputs between devices, 110× the traffic, and run 3.4–5.3× slower than a layout that exposes the device as a batch axis of the gather (§3.3). Within a device, the reduction must also stay on the core that owns each token: summing a token's rows across cores costs 11–15%.

We present RankTier, a design for MoE prefill that keeps the program static and moves the decisions it cannot compile into data. Each SoC ranks its 32 experts by load at run time and assigns them to fixed-capacity tiers by rank; the compiled program fixes the tiers, their lanes and capacities, and the run time decides which expert each lane serves. Capacities bound to ranks can be small because the order statistics of per-device load are bounded and stable across workloads, while the identities of the busy experts are not (Figure 1). Expert outputs are aggregated in a layout that preserves device ownership and assigns each token's sum to one core, and that does not depend on the tiers' shapes. Both parts need only static shapes, a sort and gathers with run-time indices.

We implement RankTier as graph transformations of the exported Qwen3-30B-A3B and evaluate all 48 layers with MXFP6 expert weights on the four SoCs of one Cloud AI 100 Ultra card, with the KV cache on device. RankTier prefills 128-, 256- and 512-token chunks in 105.3, 141.5 and 225.3 ms, 4.74×, 3.65× and 4.68× faster than the production compilation and 2.37×, 3.55× and 4.56× faster than an expert-parallel program with the exported combine. With ranking and weight indirection matched, the ranked capacities give 1.10–1.37×; fixed-size blocks under the same compiler are 1.26–1.71× slower. Capacities calibrated in advance overflowed in none of 1872 evaluation chunks of eight workloads, and a forced overflow is recovered bit-exactly. We separate what transfers to other statically partitioned targets from the behavior of this compiler (§7).

The paper makes three contributions:

- A characterization of MoE prefill under static partitioning: padded expert rows are executed work, capacity savings pay only with efficient aggregation, and graphs with identical arithmetic place that aggregation very differently (§3).
- A design that adapts expert work and preserves the aggregation structure: capacities bound to per-device load ranks, with exact overflow recovery, and token aggregation that keeps device ownership and token-local reduction through compilation (§4, §5).
- A full-model evaluation that establishes the capacity–aggregation interaction with controlled ablations, equivalent-arithmetic placement experiments and an attributed optimization ladder (§6).

2 Background and Related Work

2.1 MoE Inference

An MoE layer replaces the feed-forward block with E experts and a router. For a token with hidden state $x_t \in \mathbb{R}^d$, the router selects the set $\mathcal{K}_t$ of k experts with the largest gate values and returns

$$y_t = \sum_{e \in \mathcal{K}_t} w_{t,e}\, W_e^{\text{down}}\big(\text{SiLU}(W_e^{\text{gate}} x_t) \odot W_e^{\text{up}} x_t\big), \quad (1)$$

with normalized gate weights $w_{t,e}$. Qwen3-30B-A3B uses $E = 128$, $k = 8$, $d = 2048$ and an expert width of 768, so an expert's three matrices hold 9 MiB in FP16. Implementations group assignments by expert, run each expert's GEMMs over its gathered token rows, and return the weighted rows to their tokens; under expert parallelism the experts are spread over devices and dispatch and combine become cross-device exchanges (Lepikhin et al., 2021; Rajbhandari et al., 2022). Training systems bound each expert's work by a capacity factor and drop the tokens beyond it (Lepikhin et al., 2021; Fedus et al., 2022); at inference, a dropped assignment changes the model's output. Prefill processes the prompt in chunks of T tokens and writes their keys and values to the KV cache; decode then generates one token per step (Kwon et al., 2023; Agrawal et al., 2024). A chunk gives each layer $8T$ assignments, $2T$ per SoC on average when four SoCs hold 32 experts each.

2.2 MoE Systems

GPU systems handle the data-dependent shape of expert work directly: MegaBlocks computes experts as block-sparse GEMMs and never drops tokens (Gale et al., 2023), and DeepSpeed-MoE, FasterMoE and Lina optimize or schedule the all-to-all exchange (Rajbhandari et al., 2022; He et al., 2022; Li et al., 2023). The combine has been made sparse on GPUs too: Tutel replaces GShard's dense dispatch and combine einsums with sparse encode and decode kernels that assign tokens to thread groups (Hwang et al., 2023), and vLLM sums each token's top-k expert rows in a single kernel, with fixed-size buffers and device-side indices that suit CUDA graphs (vLLM Project, 2026; NVIDIA, 2026). A kernel author decides there which thread reads which rows and where partial sums live; a

graph compiler decides that for the program, which is the gap §3.3 examines. Placement studies exploit routing structure (Yao et al., 2024; Yu et al., 2025), Huang et al. (2024) replace static capacity with dynamic gating, and expert-choice routing fixes each expert's tokens by construction but changes the model (Zhou et al., 2022). On Trainium, expert work is packed into fixed-size blocks whose weight loads are shared across an expert's blocks and whose unused blocks are skipped (Amazon Web Services, 2026); we compare against blocks in §6.6. DynaMoE recompiles a static-shape program when capacities change during training (Kossmann et al., 2022); our full-model programs take 7–26 minutes to compile, so capacities cannot follow each chunk by recompilation.

2.3 AOT-Compiled Inference Accelerators

A Cloud AI 100 SoC has 16 AI cores and 136 GB/s of LPDDR4x bandwidth to up to 32 GB (Chatha, 2021; Qualcomm Technologies, 2025); each core has a scalar VLIW processor, a vector unit, a tensor unit with 4K FP16 MACs and 8 MB of software-managed vector memory. The Ultra card carries four SoCs under a shared 150 W limit.

The compiler takes an ONNX graph and one shape specialization per chunk length and produces a static program, partitioned over the cores of a SoC and, by multi-device tensor slicing, over the SoCs. Matrix weights can be MXFP6 (Rouhani et al., 2023), dequantized by the vector unit while the tensor unit computes. A KV cache stays on device when each cache input can be paired with its updated output (*retained state*). An ONNX `If` with a non-constant condition is rejected; dynamic-shape compilers for CPUs and GPUs handle such workloads with run-time dispatch (Shen et al., 2021; Zhu et al., 2021), which these programs lack.

3 Motivation

We study the MoE layers of Qwen3-30B-A3B compiled for the four SoCs of one Cloud AI 100 Ultra card (setup in §6.1). The exported graph (Qualcomm Innovation Center, 2024) runs a layer's 128 experts as two stages of 64 *lanes*; a lane is one expert's gate, up and down GEMMs over a fixed number of gathered token rows, its capacity C. The production program uses $C = T$, the only capacity that never drops a token. Three questions motivate RankTier: why capacities should follow the load at run time, why that is not enough without efficient aggregation, and why the graph's expression of the aggregation decides its cost.

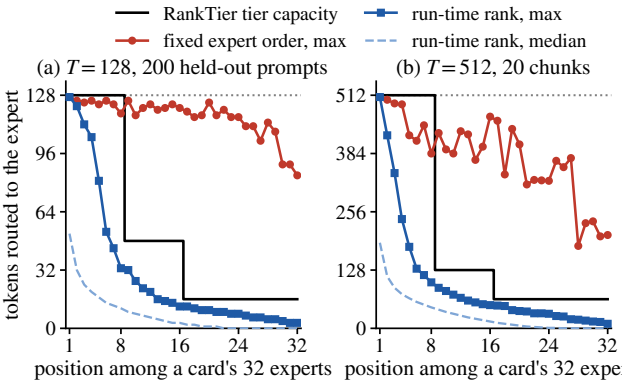


Figure 1. Rows a SoC must reserve for its expert at each position, as the largest load observed there. Red: experts in a fixed order, ranked by their mean load on calibration prompts (a: the first half of each workload's prompts, evaluated on the other half; b: leave-one-out over the chunks). Blue: experts ordered at run time by their load in the chunk itself. Black: RankTier's tier capacities. Dotted: capacity T.

3.1 Expert Loads Vary, and Padding Is Executed Work

We captured the routing of all 48 layers for 50 prompts of 128 tokens from each of eight workloads, GSM8K, MMLU, SWE-bench Lite issues, HumanEval and its JavaScript version, Alpaca instructions, CNN/DailyMail articles and Chinese XNLI sentence pairs (Cobbe et al., 2021; Hendrycks et al., 2021; Jimenez et al., 2024; Chen et al., 2021; Muennighoff et al., 2024; Taori et al., 2023; Hermann et al., 2015; Conneau et al., 2018), and for chunks of 256 and 512 tokens from four of them (Appendix C). The load is skewed, with one expert of some layer receiving 127 of 128, 256 of 256 and 508 of 512 tokens, and the skew is workload-specific: across workloads the 16 most loaded experts of a layer agree with a Jaccard index of 0.00–0.33, against 0.52–0.88 between the two halves of one workload.

A program that fixes which lane serves which expert must reserve at each lane the largest load its expert can receive. Ordering each SoC's 32 experts by their mean load on calibration prompts, the held-out maximum is 84–127 of 128 rows at *every* position (Figure 1): a drop-free fixed layout needs 91% of the rows of padding everything to T. Ordered by their load in the chunk itself, the same experts need far less below the first ranks: the 16th most loaded expert of a SoC never received more than 12 of 128 tokens.

Allocated rows matter only if they are executed. The compiler has no data-dependent control flow, but it predicates each row tile of a lane on whether the tile holds a routed token (§4.6): a lane with n tokens and row tile R computes $R\lceil n/R \rceil$ rows, and skips its weights only when it is a single tile. At $T = 512$ the static capacity-T program computes 13.8–15.4k of its 16.4k rows per SoC at the default tile size

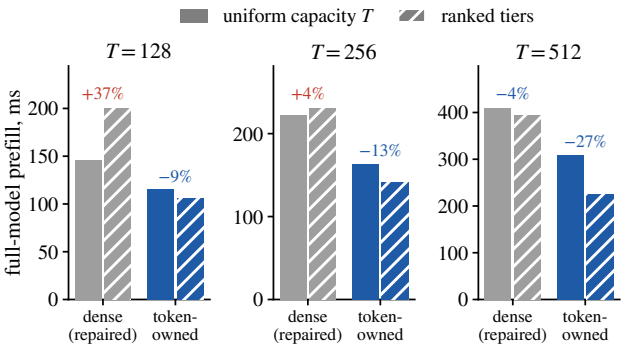


Figure 2. Capacity × aggregation. Full-model prefill latency with uniform capacity T and with RankTier's ranked tiers, under the repaired dense combine and under token-owned aggregation. All four programs rank experts at run time, gather their weights from the same bank and use the same final sum and tile size; percentages are latency changes from tiering.

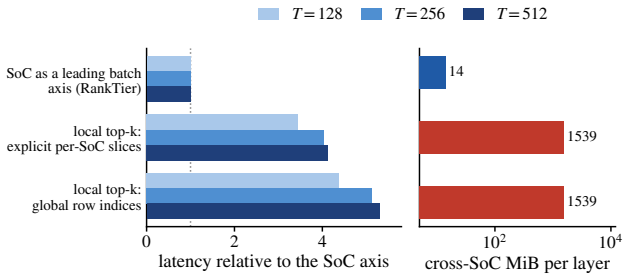


Figure 3. Equivalent arithmetic, different placement. Three expressions of the same per-SoC token aggregation, with bit-identical logits. Left: full-model latency relative to the layout-preserving SoC axis. Right: cross-SoC traffic inside one MoE layer at $T = 512$, from instrumented two-layer profiles.

and still 3.5–3.8k at the smallest one the compiler accepts. Executed padding costs time once the chunk is long: the tensor-unit time of a SoC's 16 most loaded experts is 44, 310 and 921 µs per core at capacities 92, 158 and 296, 21× for 3.2× the rows, while their weight stream stays constant.

3.2 Capacity Savings Need Efficient Aggregation

After the expert GEMMs, each expert's rows return to their tokens, are weighted and summed over the token's experts and then across devices. The exported graph does this densely: each stage scatters its rows into a zero-filled accumulator of shape $[\text{lanes}, T, d]$, sums it over each SoC's lanes and sums the four SoC partials, a cost that grows with lanes $\times T$ whatever the routing. Ranked capacities change this step's inputs: tiers of different widths produce differently shaped outputs in more stages, and a lane-dense accumulator is needed per width.

Figure 2 holds everything else fixed and varies only the capacities and the aggregation. The dense baseline is repaired: its accumulators are not read back after zero-filling, and its lane sums run in token tiles (§5). Under it, ranked tiers change full-model latency by $+37\%$, $+4\%$ and -4% at $T = 128$, 256 and 512; under token-owned aggregation, by -9%, -13% and -27%. The rows the tiers save are the same in both. Minimizing padded expert work is therefore an incomplete objective: capacity selection must be designed together with aggregation. The repaired dense combine does gain from tiers at $T = 512$, so the claim is about the size and consistency of the benefit, not about whether any benefit exists.

3.3 Equivalent Arithmetic, Different Placement

The aggregation RankTier needs is a standard sparse combine: each device gathers the rows of the experts it computed for each token, sums them into a per-token partial, and the partials are summed across devices (§2). A graph compiler, though, decides from the graph's form where each part runs. Figure 3 compares three graphs that compute exactly this sum, with the same index values, summation order and final sum, and produce bit-identical logits; they differ only in how they express which SoC owns which expert rows. Addressing each SoC's rows by global row indices into the stage's unsplit outputs, or cutting each SoC's lanes out with explicit slices, makes the compiler move expert outputs between SoCs, 1539 MiB per MoE layer against 14 MiB, and runs the full model 4.4–5.3× and 3.4–4.1× slower. A reshape that exposes the SoC partition the compiler already applies to the lane axis as the leading batch axis of the gather keeps every row on its device. With arithmetic and outputs held constant, the alternative expressions cause far more redistribution and much higher latency; they may also change scheduling, buffering or tiling, which the experiment does not separate.

Ownership is the first requirement; the second is local reduction. A token's expert rows can sit on several cores of a SoC, and a formulation that keeps device locality but sums over the lanes of every token tile pulls its operands from all 16 cores, 242 MiB of cross-core traffic per layer, and is 11–15% slower than summing each token's rows on the core that owns the token (§6.4). Expressing a top-k gather is therefore an incomplete implementation: the graph must preserve device ownership when it selects expert outputs, and assign each token's reduction to one core.

4 RankTier

Section 3 leads to three requirements. First, capacity must follow each chunk's load without control flow: the program must choose at run time which expert a lane serves, and bind capacities to a quantity that is stable across chunks. Second, aggregation must stay efficient whatever shapes the capacities take: keep each device's expert rows on that

device, sum each token's rows on one core, and cost in proportion to the routed rows, not to lanes $\times\, T$. Third, both must be static graphs of operations that ahead-of-time targets provide: reductions, a sort, gathers with run-time indices and one cross-device sum. This section states the method independent of the accelerator; §5 summarizes how the Cloud AI 100 compiler shaped its realization.

4.1 A Static Model of an MoE Layer

A device c owns a fixed set $\mathcal{E}_c$ of experts, 32 of the 128 on each SoC here. A static program computes them in *lanes*: a lane is one expert's three GEMMs over C token rows gathered through a token table, and lanes of equal capacity that run together form a stage. The boundary between compile time and run time is sharp. Fixed at compile time: the number of tiers, their lanes and capacities, the shapes of every tensor, the weight bank and the expert-to-device placement. Decided by data at run time: which expert each lane serves, which tokens fill its rows, which weights it gathers, and which rows each token's aggregation reads. A chunk routes n_e of its kT assignments to expert e and $N_c = \sum_{e\in\mathcal{E}_c} n_e$ to device c, and a lane that serves e drops tokens unless $n_e \le C$. A lane's cost has a weight part, its expert's matrices streamed from memory, and a row part that grows with the rows it computes: all C of them without control flow, or $R\lceil n_e/R\rceil$ on a target that predicates row tiles of R rows (§4.6). Either way the capacity, fixed before the routing is known, bounds a lane's work.

4.2 Rank-Bound Capacities

After the router, each device counts its experts' tokens and sorts the counts, which gives a permutation π_c from rank to expert with $n_{\pi_c(1)} \ge n_{\pi_c(2)} \ge \dots$. The device's lanes form tiers: tier i is a stage of L_i lanes at capacity C_i that covers ranks $a_i, \dots, a_i + L_i - 1$, with $\sum_i L_i = |\mathcal{E}_c|$, and lane j of tier i serves expert $\pi_c(a_i + j)$. The lane gathers that expert's weights through π_c and its token table from tables that the routing computes for all experts in their fixed order (Figure 4). Every expert keeps exactly one lane, so a tiering changes only the padding: the same experts compute the same tokens, and as long as every expert fits its lane, the output equals that of any other tiering, capacity T everywhere included (bit-identical in our implementation). A device provisions $\sum_i L_i C_i$ rows per layer against $|\mathcal{E}_c|\,T$.

Why ranks. The order statistics of per-device load are bounded for a structural reason. The r most loaded experts of device c each hold at least $n_{\pi_c(r)}$ assignments, and together at most N_c, so

$$n_{\pi_c(r)} \;\le\; N_c/r, \qquad N_c \;\le\; \min(k, |\mathcal{E}_c|)\,T. \tag{2}$$

N_c sums over all of the device's experts, $kT|\mathcal{E}_c|/E = 2T$ on average here and at most $1.9\times$ that in our 256- and 512-token captures, so every rank beyond the first few is bounded well below T whatever the workload. A capacity attached to an expert must instead cover the worst case of that one expert, which approaches T because routing is skewed and which experts are hot changes with the workload (§3.1). Eq. (2) holds for any routing; that the per-rank maxima are also stable across workloads, while expert identities are not, is a property of the model, which we measured on held-out documents of eight workloads (Figure 1, §6.8).

4.3 Calibrating the Tiers

Capacities. The first tier has capacity T, since rank 1 can receive every token. For $i > 1$, C_i is the largest load observed at rank a_i, the tier's first rank, over calibration chunks, times a headroom h, rounded up to the target's row granularity g and capped at T; since ranks are sorted, rank a_i bounds the whole tier. We calibrate with $h = 1.28$ and $g = 16$ on routing captures of the eight workloads: the 128-token prompts of §3.1, 56 chunks of 256 tokens and 20 of 512.

Structure. The number and widths of the tiers trade padded rows against per-stage cost. Rows alone favor many tiers: under our calibration at $T = 256$ and 512, three tiers need 65–67% of the rows of a two-tier split at T and $T/8$, four 52–58% and five 48–56%. But each tier is a stage with a cost of its own, a weight pipeline and one aggregation gather, and a target may restrict the widths a stage can take. We fix the structure by measurement, not by a model: on Cloud AI 100, where widths must divide 16 and stages run one after another (§5), we compared two, three and four tiers and deploy three tiers of 8, 8 and 16 lanes per SoC at every chunk length, with capacities T, C_2 and C_3 of 128/48/16, 256/64/32 and 512/128/64 for $T = 128$, 256 and 512: 12–13T rows per SoC against $18T$ for two tiers and $32T$ for capacity-T padding. This structure is not optimal everywhere: at $T = 256$ two tiers are 2.5% faster in the full model, although three save a third of the rows (§6.5). Choosing the structure per chunk length from measured stage costs remains open.

4.4 Overflow Detection and Recovery

The lower tiers are calibrated, not dropless, so a deployment must detect overflow and recover from it exactly. Detection is free: the program already computes every lane's token count, and the host compares it with the lane's capacity after each chunk. Recovery reruns the chunk on a capacity-T variant at the same positions. Prefill writes the KV cache only at the chunk's own positions, so the rerun overwrites every entry the failed attempt wrote, later chunks see only the rerun's cache, and the result is that of an overflow-free run. The variant can be a second profile

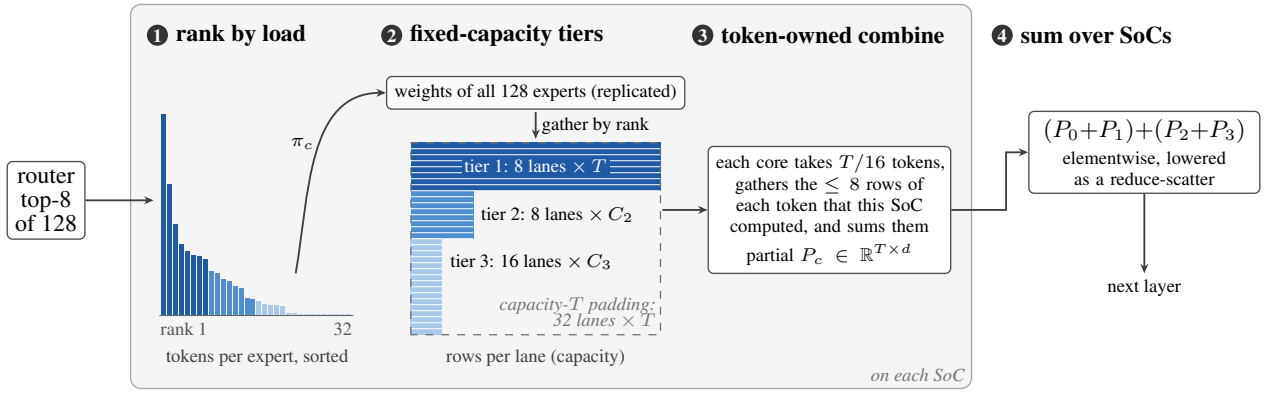


Figure 4. One MoE layer under RankTier. (1) Each SoC counts the tokens routed to its 32 experts and sorts them (bars: SoC 2 in layer 20 of the held-out 512-token chunk; colors mark the tiers). (2) Three tiers of lanes with compile-time capacities ($C_2 = 128$, $C_3 = 64$ at $T = 512$) serve the experts by rank; their weights are gathered from a bank of all 128 experts through the rank permutation π_c. The filled area is the padded work, against the dashed capacity-T layout. (3) Each core gathers and sums the rows of its own tokens. (4) One elementwise sum over SoCs completes the layer.

of the same program, since tier capacities read from the lengths of small row inputs let the compiler specialize one program per profile over the same weights and cache, or a second program kept resident, which must first rebuild the cache from the earlier chunks. With overflow probability p a chunk costs $t_{\text{tier}} + p\,t_T$ on average, so the tiered program's own latency matters most (§6.7).

4.5 Ownership-Preserving Token Aggregation

After the expert GEMMs, each token's k weighted expert rows must be summed. Device c computes the experts $\mathcal{E}_c$, and the result is

$$P_c[t] = \sum_{j:\, e_{t,j} \in \mathcal{E}_c} O_c[\text{row}(t,j)], \qquad y[t] = \sum_c P_c[t], \quad (3)$$

where O_c holds device c's weighted expert outputs and $\text{row}(t,j)$ addresses the output of token t's j-th expert. The arithmetic is the standard local top-k reduction; what RankTier contributes is a graph form that a compiler executes locally and in parallel, independent of the tiers' shapes (Algorithm 1). It makes three choices.

Ownership as layout. A stage or tier computes its L lanes as one output of shape $[L, C, d]$, whose lane axis the compiler already partitions across the D devices, L/D lanes each. RankTier reshapes it, without moving data, to $[D, (L/D)\,C, d]$, which makes the device the leading batch axis of one batched gather with per-device row indices of shape $[D, Tk]$: for token t's j-th expert e, the index on its own device is $l\,C + \text{slot}(e,t)$, with l the expert's lane on that device, and every other device gets a safe index and a zero mask. Global row indices into the flattened outputs, or explicit per-device slices, address the same rows, but with them the compiler we use moved expert outputs between devices (§3.3).

Token-local reduction. The gathered rows, $[D, Tk, d]$, are summed over k into the partials $[D, T, d]$. The compiler splits this sum by token tiles, so each core gathers and adds the at most k rows of its own tokens, with no cross-core reduction. A lane-indexed formulation, which gathers every lane's row for each token and sums over lanes, keeps device locality but pulls each reduction's operands from all cores (§6.4).

Elementwise cross-device sum. The partials are summed as $(P_0 + P_1) + (P_2 + P_3)$ per token tile, which the compiler lowers as a reduce-scatter with no root device; the next layer's input normalization completes it.

The indices need nothing beyond the routing. The router's top-k output lists each token's experts; integer arithmetic on the placement and small per-tier tables map an expert to its (tier, device, lane), and the per-expert slot tables that the routing computes anyway give its row.

Cost. Per device and tier the gather produces Tk rows of d elements, masked slots included, against $L\,T$ rows of a lane-dense accumulator, and a tier adds one such gather whatever its width and capacity; the cross-device traffic is one $[T, d]$ partial per device. A lane-dense combine instead needs one accumulator per tier width, which grows with T and absorbs the rows the tiers save (Figure 2).

4.6 Capacities and Tile-Granular Skipping

A compiler may already skip padded work at run time without control flow. Cloud AI 100 lowers the export's mask of empty slots, $\text{Where}(r < n_e,\ y,\ 0)$, into a predicate per row tile of a lane: the tile's gather, GEMMs and SiLU run only if it holds a routed token, so a lane computes $R\lceil n_e/R \rceil$ rows for tiles of R rows, and an empty expert's weights are skipped only when its lane is a single tile (Appendix G).

Algorithm 1 Ownership-preserving token aggregation for one tier s (D devices)

Require: top-k experts $\mathcal{K}[t, 1..k]$ of each token t; map $\ell(e) \mapsto (s, c, l)$ (tier, device, lane); slot table $\mathrm{slot}[e, t]$; tier outputs O_s of shape $[L_s, C_s, d]$, lanes partitioned over devices

1: $\hat{O}_s \leftarrow \mathrm{reshape}(O_s, [D, (L_s/D)\, C_s, d])$ {no data movement}
2: **for** all $t \le T$, $j \le k$, devices c **in parallel do**
3: $\quad e \leftarrow \mathcal{K}[t, j]$; $(s', c', l) \leftarrow \ell(e)$
4: $\quad m[c, t, j] \leftarrow [s' = s \wedge c' = c]$; $idx[c, t, j] \leftarrow m \,?\, l\, C_s + \mathrm{slot}[e, t] : 0$
5: **end for**
6: $G \leftarrow \mathrm{gather}(\hat{O}_s, idx) \odot m$ {batched over devices: $[D, Tk, d]$}
7: $P_s \leftarrow \sum_j G[\cdot, \cdot, j]$ {per token, on its core: $[D, T, d]$}
8: **after all tiers:** $P \leftarrow \sum_s P_s$; $y \leftarrow (P_0 + P_1) + (P_2 + P_3)$ {elementwise}

The tile follows the compiler's maximum tile size. Such skipping is coarse and does not shrink what a capacity provisions: token tables, buffers and the weight reads of multi-tile lanes stay at capacity. Rank-bound capacities compose with it: they set the lane size, and with it the tile, per rank, so the low ranks get single-tile lanes that skip empty experts entirely. Every program therefore runs at its own best tile size (§6.3).

5 Realization on Cloud AI 100

We realize RankTier as ONNX graph transformations of the Qwen3-30B-A3B graph that the vendor's QEfficient library exports for Cloud AI 100 (Qualcomm Innovation Center, 2024), about 1,200 lines of Python with one pass per step of the evaluation's ladder (§6.1). The compiler shaped six choices, detailed in Appendix B. (i) *Expert parallelism with an on-device cache:* forbidding the partitioner to cut weights across SoCs puts whole experts on each SoC, one lane per core, and cuts the layer's cross-SoC traffic from 121.6 to 1.5 MiB, but it also stops the KV cache from staying on device; head-parallel attention, one KV-head group per SoC, restores it. (ii) *The attention's group sum:* the four head groups' partial outputs must be summed elementwise; written as an einsum, the sum took 31% of each layer at $T = 512$. This repair belongs to attention, and the evaluation attributes it separately. (iii) *One gathered bank:* weights gathered with a run-time index cost what static weights do (a one-layer replay runs in 2.796 ms either way), because the gather becomes an indirect DMA of the MXFP6 bytes; but only a gather from one constant holding all 128 experts stays this fast, so the bank is replicated, 22.5 GiB per SoC instead of 6.7 GiB. (iv) *Tier widths:* a stage maps evenly onto a SoC's cores only when its lanes divide 16, and the compiler runs a layer's stages one after another, so widths come from $\{1, 2, 4, 8, 16\}$ and every tier adds a sequential window. (v) *The sort off the critical path:* the routing tables are computed for all experts in their fixed order, and only a row permutation and the weight gathers wait for the sort; the maximum tile size is chosen per program and chunk length (Appendix E). (vi) *The aggregation's graph form:* among equivalent expressions, only the layout-preserving SoC axis kept every expert row on its SoC (§3.3). Tree prefix scans for the routing and two repairs of the exported dense combine remove serialized work before the combine is replaced; the evaluation keeps them as steps of their own. Each chunk length is its own program, with sequence length T, context $2T$, MXFP6 matrix weights and FP16 activations (Appendix A).

6 Evaluation

We attribute RankTier's speedup (§6.2), examine how capacities and aggregation interact (§6.3–§6.5), and test blocks, multiple chunks, generalization, accuracy and a fused kernel (§6.6–§6.10).

6.1 Experimental Setup

Hardware and model. One Cloud AI 100 Ultra card, four SoCs with 16 cores and 32 GB each, platform SDK 1.21.6; Qwen3-30B-A3B, all 48 layers, batch 1, MXFP6 matrix weights and FP16 activations as in the production deployment, and the KV cache on device.

Programs. The ladder of Table 1 adds one step at a time: (1) the production compilation of the exported graph; (2) expert-parallel compilation with head-parallel attention, the export's combine and capacity T, our *naive* T/T baseline; (2′) attention's sum over its head groups written elementwise, kept by every later program; (3a) tree prefix scans for the routing; (3b) two repairs of the dense combine; (4) token-owned aggregation and (5) its elementwise cross-SoC sum (§4.5); (6) the run-time rank sort with two tiers at T and $T/8$; (7) RankTier's three tiers. Controlled variants change one choice of these programs.

Inputs and protocol. At $T = 128$ the first chunk of a GSM8K chat prompt, at 256 and 512 held-out GSM8K chunks outside the calibration range; latency depends on a chunk only through its routing, and on one layer it varied by about 1% across 200 prompts of four workloads. A pass is the median of three rounds of 20 inferences after warm-up. Sessions time two to six programs in two or three passes of mirrored order, always with program 5 as an anchor, and one program + session fit of log latency over all passes (66 to 75 per chunk length) gives each program's latency; the largest residual is 0.50%, 0.88% and 1.7% at $T = 128$, 256 and 512. We compare the last-position logits with an FP32

Table 1. Prefill latency of one chunk (ms) along the ladder, the change against the step above, and the contribution of each group of steps.

Program	$T = 128$	$T = 256$	$T = 512$
1 Production	499.6	516.5	1053.5
2 + expert parallel	266.9 −46.6%	569.3 +10.2%	1157.6 +9.9%
2′ + head-group sum	250.0 −6.3%	502.4 −11.8%	1027.4 −11.2%
3a + tree scans	212.4 −15.1%	431.0 −14.2%	877.8 −14.6%
3b + dense repairs	141.9 −33.2%	289.9 −32.7%	570.9 −35.0%
4 + token-owned	115.8 −18.5%	189.9 −34.5%	350.9 −38.5%
5 + elementwise sum	113.2 −2.2%	184.8 −2.7%	349.2 −0.5%
6 + rank sort, 2 tiers	106.3 −6.1%	138.0 −25.3%	269.6 −22.8%
7 + 3 tiers (RankTier)	**105.3** −0.9%	**141.5** +2.5%	**225.3** −16.4%
Aggregation, 3a→5	1.88×	2.33×	2.51×
Capacities, 5→7	1.07×	1.31×	1.55×
vs. T/T, 2′ →7	2.37×	3.55×	4.56×
vs. production, 1→7	**4.74**×	**3.65**×	**4.68**×

CPU run and read back every lane's count.

6.2 End-to-End Results

Table 1 gives the results (Figure 5 in Appendix F plots them). RankTier prefills 128-, 256- and 512-token chunks in 105.3, 141.5 and 225.3 ms, 4.74×, 3.65× and 4.68× faster than the production compilation and 2.37×, 3.55× and 4.56× faster than naive T/T. The attention rewrite (2→2′) saves 17, 67 and 130 ms in every head-parallel program alike; it is not an MoE gain. Without it, naive expert parallelism is 10% slower than production at $T = 256$ and 512; with it, 2.5–2.7% faster, and 2.00× faster at $T = 128$, where production takes about as long as for 256 tokens. The routing scans (3a) give 1.17–1.18×. MoE output aggregation, steps 3b–5, is the largest factor at 1.88×, 2.33× and 2.51×, and the ranked capacities add 1.07–1.55×. Only with both does a longer chunk pay: per token, RankTier costs 0.82, 0.55 and 0.44 ms at $T = 128$, 256 and 512, while naive T/T stays at 1.95–2.01 ms.

6.3 Capacities and Aggregation Together

The interaction. Figure 2 isolates it: all four programs rank experts at run time, gather weights from the same bank and share the final sum and the tile size. The repaired dense combine keeps one zero-filled accumulator per tier width, never reads its zeros back, lets a width group's later tiers read, add and scatter back, and sums lanes in 16 token tiles. With it, tiers change latency by +36.9%, +3.9% and −3.6%; with token-owned aggregation, by −8.8%, −13.3% and −26.9%. Token-owned aggregation is 1.27–1.36× faster than the dense combine at capacity T, and 1.63–1.90× with tiers.

Attributing the capacities. Steps 5–7 change weight access, the assignment of experts to lanes and the capacities at once. Table 2 separates them with five programs that share the aggregation: (A) a fixed expert order at capacity T with static weights; (B) the same order through the bank gathers; (C) the run-time rank at capacity T; (D) two tiers; (E) RankTier. All give bit-identical logits. With ranking and indirection matched, RankTier's capacities are 1.10, 1.15 and 1.37× faster (C→E), and RankTier is 1.08–1.28× faster than the best static program. Each program runs at its best tile size, the granularity at which the compiler skips padded rows (§4.6, Appendix G): with 512 KiB tiles the static program computes a quarter of its rows, while the gathered programs spill activations at that size and use 1024 KiB tiles at $T = 512$.

Table 2. Capacity attribution (ms per chunk, each program at its best tile size), and expert rows provisioned per SoC.

	$T = 128$	$T = 256$	$T = 512$
A static, fixed order, T	113.8	168.7	288.0
B gathered, fixed order, T	109.1	160.0	302.3
C gathered, ranked, T	115.5	163.1	308.1
D ranked, two tiers	106.3	**138.0**	269.6
E RankTier, three tiers	**105.3**	141.5	**225.3**
C/E, capacities	1.10×	1.15×	1.37×
A/E, vs. best static	1.08×	1.19×	1.28×
Rows, A–C/E	4096/1664	8192/3072	16384/6144

Table 3. Aggregation formulations at capacity T with static weights and the same routing: full-model latency relative to RankTier's aggregation (ladder step 5), cross-SoC MiB per MoE layer at $T = 512$ (layer 1 of instrumented two-layer cuts), and whether the logits equal step 5's or differ in FP16 summation order.

Formulation	128	256	512	MiB	Logits
Step 5: SoC batch axis	1.00	1.00	1.00	14	–
Local top-k, SoC slices	3.44	4.04	4.14	1539	equal
Local top-k, global rows	4.37	5.14	5.30	1539	equal
Top-k rows from all SoCs	4.05	4.70	4.92	1538	order
Lane-indexed, lane axis	1.12	1.11	1.15	12	order
Lane-indexed, SoC slices	7.02	8.22	4.31	577	order
Repaired dense combine	1.25	1.57	1.64	48	order

6.4 What Makes the Aggregation Efficient

Table 3 compares RankTier's aggregation, ladder step 5, with six formulations at the same capacity and routing. The two local top-k variants compute exactly its sum in its order and differ only in how the graph expresses device ownership; both make the compiler ship expert outputs between SoCs, as does the GPU-style gather of each token's k rows from all SoCs (§3.3). The repaired dense combine keeps locality, but its accumulator grows with T: replacing the representation (step 4) gives 1.23–1.63× on top of the 1.49–1.54× of the repairs (3b).

Local reduction. The lane-indexed formulation keeps de-

vice locality, with slightly less cross-SoC traffic than step 5 (12 against 14 MiB), and is still 11–15% slower. It sums each token tile over a SoC's 16 lanes, which sit on 16 cores: in the profile this sum moves 242 MiB between cores per layer and keeps them busy for 47 core-ms, against 1.9 MiB and 2.4 core-ms for step 5's sum over each token's k rows on one core, and its 16 tiles run largely one after another. The window from the last expert GEMM to the end of the final sum grows from 1.2–1.7 to 3.1–3.7 ms per SoC (Appendix H); the uninstrumented full models differ by 1.1 ms per layer, so these windows locate the difference rather than decompose it along a dependency chain.

6.5 Tier Structure

Each tier is a stage with a cost of its own. In a full-model profile of RankTier at $T = 512$ (before the attention fix), the 8×128 and 16×64 tiers span 0.59 and 0.52 ms per layer while their GEMMs keep the busiest core busy for 0.08–0.09 ms: they wait on their experts' weights, and each tier starts after a gap of 0.28–0.48 ms. The first tier splits because it is bound by the SoC's memory traffic: 16 lanes at capacity 512 take 2.2 ms, eight at 512 and eight at 128 take 0.8–1.0 and 0.65–0.72 ms (Appendix L). Three tiers save 28–33% of the rows of two, and their worth depends on the chunk: 1.20× at $T = 512$, 1.01× at 128 and 0.975× at 256 (Table 2), where a session before the attention fix had found them 3.6% faster; four tiers lost to three on two-layer cuts (Appendix K). Rows do not predict the best structure, and the deployed one is not the best at every chunk length; selecting it from measured stage costs remains open.

6.6 Fixed-Size Blocks Instead of Ranked Lanes

Block scheduling packs assignments into blocks of B rows, one expert per block, with $\lceil kT/B \rceil + E - 1$ blocks to drop nothing (Amazon Web Services, 2026). Built from RankTier's own components and measured on two-layer cuts before the attention fix (Appendix J), the best calibrated block program is 1.26, 1.39 and 1.69× slower than RankTier at $T = 512$, 256 and 128, the best dropless one 1.51–1.71×, and even a trace-specialized diagnostic 1.12–1.31×. Blocks lose on weight traffic: every block slot streams its expert's MXFP6 weights, used or not, 337.5 MiB per SoC and layer for 64-row dropless blocks against 95–106 MiB for RankTier, and one weight load cannot serve consecutive blocks of an expert in this compiler.

6.7 Beyond the First Chunk, and Overflow

The programs keep the cache for $2T$ positions. A second chunk attends to the cache the first wrote: RankTier and the static capacity-T program give bit-identical logits for both chunks at all three lengths, and the second chunk costs what the first does. To test the recovery of §4.4, we forced an undersized capacity profile on the second of four 128-token chunks: the counts flagged 45 (layer, lane) pairs, the rejected logits were 0.04 relative L_2 off, and after the rerun all four chunks were bit-identical to runs without overflow, as after an overflow in the first chunk. Capacity inputs and extra profiles make the tiered program 18% slower, so a resident capacity-T program may be the cheaper fallback, at the cost of re-prefilling earlier chunks, which we did not time. These tests predate the attention fix.

6.8 Do Calibrated Capacities Generalize?

We split each of eight workloads into calibration and evaluation documents before cutting chunks, and measured the routing of every chunk on the device. RankTier's capacities overflowed in none of the 805, 618 and 449 evaluation chunks at $T = 128$, 256 and 512 and in no workload (30 chunks hold documents the deployed capacities were calibrated on), nor did leave-one-workload-out calibration; without headroom, the rule gives a 112-row middle tier at $T = 512$, which overflows in 2 of 449 chunks. The same tiers bound to a fixed expert order need capacity T everywhere; calibrated per workload, they need 1.4–2.7× the rows and still overflow in up to 30% of chunks, while ranks 1, 9 and 17 never exceeded 509, 119 and 52 tokens. The bound $n_{(r)} \le N/r$ holds with room (at most 0.97 of it).

6.9 Accuracy

Every program stays at the error level of the MXFP6 weights: the relative L_2 error of the last-position logits against FP32 is 0.188, 0.091 and 0.096 for RankTier at $T = 128$, 256 and 512, against 0.188, 0.089 and 0.095 for production (Appendix M). The programs differ from production only in FP16 summation order, in attention's group sum and in the order of a token's expert rows; A–E are bit-identical to each other. The next token of RankTier and of production matches FP32 except in the 256-token chunk, whose two leading FP32 logits are 0.58 apart. No lane exceeded its capacity.

6.10 A Fused Kernel in the Same Layout

TokenCombine, a C++ custom operator with HVX vector intrinsics, replaces step 5's gather, mask and sum over the same $[4, 16C, d]$ layout (Appendix I); its tiles follow the SoC axis over all 64 cores, and cross-SoC traffic stays at step 5's 14 MiB. At $T = 512$ it runs at 0.940× of step 5 in the fit (323.3–324.7 against 341.9–343.8 ms in one alternating session). Its logits differ deterministically from step 5's by 1.7% relative L_2, because it sums in FP32 in another order, and it takes 650–656 minutes to compile. The layout thus also supports a fused implementation, with a modest gain at the one chunk length we tested.

Table 4. What transfers: for each part of RankTier, the reasoning we expect to hold on other statically partitioned targets, the conditions under which it pays, and the Cloud AI 100 compiler behavior that shaped our realization of it.

Part	Transferable reasoning	Pays when	Shaped here by the compiler
Rank-bound capacities	a lane serves a load rank within a device's experts; the r-th largest of n assignments is at most n/r; per-rank maxima were stable across workloads, identities were not	expert work is fixed before routing; padded rows cost time; many experts per device	no control flow, so a sort and gathers; one gathered constant, replicated; tile size per chunk length
Tier structure	first tier at T, lower tiers at h times the calibrated maximum of their first rank; rows traded against measured per-stage cost; overflows show in the counts and a rerun recovers them exactly	a stage's fixed cost is small against the rows it removes	widths must divide 16 and stages run in sequence
Ownership-preserving aggregation	designed with the capacities; device ownership stated in a form the compiler keeps; each token's rows summed on the core that owns it; one elementwise cross-device sum; cost grows with kT per tier	the baseline combine is dense in lanes; capacities change the outputs' shapes	index values and slices lost locality, the SoC batch axis kept it; gather without zero fill

6.11 Limitations

The lower tiers are calibrated, not dropless: no evaluation chunk overflowed, but the deployed rate depends on model and workload. We evaluate one model, at batch 1 and in prefill only (our rewrites fix T), and blocks under this compiler only. The tier structure was chosen by comparison, not by a validated method, and the ownership results concern this compiler's partitioner. The SoCs share a 150 W limit under which most padded programs throttle at times (RankTier never does), so we compare programs through the anchored fit rather than across sessions.

7 What Transfers

Table 4 separates, for each part of RankTier, the reasoning we expect to hold on other statically partitioned targets from the conditions under which it pays and the compiler behavior that shaped it. We measured one model under one compiler, so this section states expectations, not results.

The method. Capacities fixed before routing is known should follow per-device load ranks, and the aggregation must be designed with them: its cost must not grow with the shapes the capacities create, and the graph must state device ownership and token-local reduction in a form the compiler preserves. The order-statistics bound holds for any routing; its stability across workloads is a property of the model.

When it pays. Padded rows must cost time: a 16-lane stage of ours streams its MXFP6 weights in about 0.6 ms, stays at that floor with 92 rows per lane and is bound by its tensor-unit time with 296, and the compiler skips padded rows only in whole tiles. Kernels that size expert work at run time have no padding to remove (Gale et al., 2023), and blocks whose weight loads are shared across an expert's blocks (Amazon Web Services, 2026) may close the gap we measured. Ranking needs many experts per device relative to k: 32 per SoC against $k = 8$ here, while 8 experts with $k = 2$ on four devices would leave little to rank.

What the compiler shaped. Which graph forms keep ownership is a property of this partitioner; another compiler may keep index values or slices local, or need another form. We expect the requirement to transfer, that the graph let the compiler see which device owns which rows, not the reshape that met it here. The attention rewrite and the dense-combine repairs fix this export and compiler, and the production baseline reflects its default partitioning, which takes about as long for 128 tokens as for 256: our speedups over production do not transfer, while the reasoning does. A scheduler that overlaps independent stages would favor more tiers.

8 Conclusion

Ahead-of-time compilation fixes, before routing is known, how many rows each expert computes and how its outputs return to their tokens. RankTier adapts the first through data, binding compiled capacity tiers to per-device load ranks, and keeps the second efficient with an aggregation graph that preserves device ownership and token-local reduction. The two must be designed together: ranked tiers cut the latency of 512-token chunks by 27% with this aggregation but by 4% with a repaired dense combine. On four Cloud AI 100 SoCs, RankTier prefills Qwen3-30B-A3B 3.65–4.74× faster than the production compilation. Selecting the tier structure from measured stage costs, and other models and compilers, come next.

References

Abts, D., Ross, J., Sparling, J., Wong-VanHaren, M., Baker, M., Hawkins, T., et al. Think fast: A tensor stream-

ing processor (TSP) for accelerating deep learning workloads. In *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, pp. 145–158, 2020.

Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., Tumanov, A., and Ramjee, R. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 117–134, 2024.

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 4895–4901, 2023.

Amazon Web Services. MoE inference architecture deep dive, AWS Neuron SDK 2.30.0. https://awsdocs-neuron.readthedocs-hosted.com/en/v2.30.0/libraries/nxd-inference/developer_guides/moe-arch-deep-dive.html, 2026. Accessed September 30, 2026.

Chatha, K. Qualcomm cloud AI 100: 12 TOPS/W scalable, high performance and low latency deep learning inference accelerator. In *2021 IEEE Hot Chips 33 Symposium (HCS)*, pp. 1–19, 2021. doi: 10.1109/HCS52781.2021.9567417.

Chen, M., Tworek, J., Jun, H., Yuan, Q., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Conneau, A., Rinott, R., Lample, G., Williams, A., Bowman, S., Schwenk, H., and Stoyanov, V. XNLI: Evaluating cross-lingual sentence representations. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 2475–2485, 2018.

DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

Gale, T., Narayanan, D., Young, C., and Zaharia, M. MegaBlocks: Efficient sparse training with mixture-of-experts. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5, 2023.

He, J., Zhai, J., Antunes, T., Wang, H., Luo, F., Shi, S., and Li, Q. FasterMoE: Modeling and optimizing training of large-scale dynamic pre-trained models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pp. 120–134, 2022.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.

Hermann, K. M., Kočiský, T., Grefenstette, E., Espeholt, L., Kay, W., Suleyman, M., and Blunsom, P. Teaching machines to read and comprehend. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.

Hillis, W. D. and Steele, Guy L., J. Data parallel algorithms. *Communications of the ACM*, 29(12):1170–1183, 1986.

Huang, H., Ardalani, N., Sun, A., Ke, L., Lee, H.-H. S., Bhosale, S., Wu, C.-J., and Lee, B. Toward efficient inference for mixture of experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.

Hwang, C., Cui, W., Xiong, Y., Yang, Z., Liu, Z., Hu, H., Wang, Z., Salas, R., Jose, J., Ram, P., Chau, J., Cheng, P., Yang, F., Yang, M., and Xiong, Y. Tutel: Adaptive mixture-of-experts at scale. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 5, 2023.

Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Kossmann, F., Jia, Z., and Aiken, A. Optimizing mixture of experts using dynamic recompilations. *arXiv preprint arXiv:2205.01848*, 2022.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pp. 611–626, 2023.

Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021.

Li, J., Jiang, Y., Zhu, Y., Wang, C., and Xu, H. Accelerating distributed MoE training and inference with Lina. In *2023 USENIX Annual Technical Conference (USENIX ATC)*, pp. 945–959, 2023.

Muennighoff, N., Liu, Q., Zebaze, A., Zheng, Q., Hui, B., Zhuo, T. Y., Singh, S., Tang, X., von Werra, L., and Longpre, S. OctoPack: Instruction tuning code large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

NVIDIA. Megatron core: vLLM-style fused MoE implementation. https://docs.nvidia.com/megatron-core/developer-guide/0.19.2/apidocs/core/core.inference.moe.vllm_fused_moe.html, 2026. Accessed October 2026.

Qualcomm Innovation Center. Efficient transformers library (QEfficient). https://github.com/quic/efficient-transformers, 2024.

Qualcomm Technologies. Cloud AI 100 architecture. Qualcomm Cloud AI SDK documentation, https://quic.github.io/cloud-ai-sdk-pages/latest/Getting-Started/Architecture/, 2025.

Rajbhandari, S., Li, C., Yao, Z., Zhang, M., Aminabadi, R. Y., Awan, A. A., Rasley, J., and He, Y. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, pp. 18332–18346, 2022.

Rouhani, B. D., Zhao, R., More, A., Hall, M., Khodamoradi, A., Deng, S., Choudhary, D., Cornea, M., Dellinger, E., Denolf, K., et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

Shen, H., Roesch, J., Chen, Z., Chen, W., Wu, Y., Li, M., Sharma, V., Tatlock, Z., and Wang, Y. Nimble: Efficiently compiling dynamic neural networks for model inference. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 3, pp. 208–222, 2021.

Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P., and Hashimoto, T. B. Stanford Alpaca: An instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

vLLM Project. MoE alignment and sum CUDA kernels (`moe_sum`). https://github.com/vllm-project/vllm/blob/main/csrc/libtorch_stable/moe/moe_align_sum_kernels.cu, 2026. Accessed October 2026.

Yang, A. et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Yao, J., Anthony, Q., Shafi, A., Subramoni, H., and Panda, D. K. Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference. *arXiv preprint arXiv:2401.08383*, 2024.

Yu, Z., Guan, Y., Yu, Z., Zhou, C., Hu, Z., Pei, S., Kang, Y., Ding, Y., and Tsai, P.-A. Patterns behind chaos: Forecasting data movement for efficient large-scale MoE LLM inference. *arXiv preprint arXiv:2510.05497*, 2025.

Zhou, Y., Lei, T., Liu, H., Du, N., Huang, Y., Zhao, V., Dai, A., Chen, Z., Le, Q., and Laudon, J. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Zhu, K., Zhao, W., Zheng, Z., Guo, T., Zhao, P., Bai, J., Yang, J., Liu, X., Diao, L., and Lin, W. DISC: A dynamic shape compiler for machine learning workloads. In *Proceedings of the 1st Workshop on Machine Learning and Systems (EuroMLSys)*, pp. 89–95, 2021.

A Compilation Flags and Program Sizes

All programs are compiled with the command below, with a four-SoC partition configuration and one shape specialization per chunk length (sequence length T, context $2T$). Every program but production (step 1) adds `-mdts-mos=1`. Steps 6–7 set the maximum tile size (`-size-split-granularity`), 512 KiB for $T \leq 256$ and 1024 KiB for $T = 512$; the attribution programs use their own best size (Table 2). The TokenCombine program registers the operator's configuration with `-register-custom-op`.

```
qaic-compile -aic-hw -aic-hw-version=ai100
  -retained-state -convert-to-fp16
  -mxfp6-matmul -aic-num-cores=16 -mos=1
  -aic-enable-depth-first
  -mdp-load-partition-config=<4 SoCs>
  -network-specialization-config=<T, 2T>
  [-mdts-mos=1]                # steps 2-7
  [-size-split-granularity=512|1024]
  [-register-custom-op=<yaml>]
```

Table 5 lists the program sizes.

Table 5. Program file, device memory per SoC and full-model compile time.

Programs	File (GiB)	Memory/SoC (GiB)	Compile (min)
Static weights (1–5, A)	25	6.6–7.1	7–26
Gathered weights (6–7)	88	22.4–22.5	9–20
TokenCombine ($T = 512$)	25	–	650–656

B Realization Details on Cloud AI 100

This appendix records how the Cloud AI 100 export and compiler shaped the realization summarized in §5.

B.1 What the Compiler Does by Default

Slicing across SoCs. With its default heuristic the partitioner splits the down-projection weights of all 64 lanes four ways along the output dimension, so each SoC computes a 512-column slice of every expert's output. This requires an all-gather of the intermediate activation (36 MiB per stage at $C = 128$) and a re-shard of the output (24 MiB per stage) before the combine; cross-SoC traffic reaches 121.6 MiB per layer, while gate and up projections stay expert-parallel.

A dense accumulator. The exported combine scatters each stage's weighted expert rows into an accumulator of shape $[64, T, d]$, zero-filled by a single core, gathered and updated between the two stages, reduced over each SoC's 16 lanes by one einsum that the compiler runs through DRAM on one core, and finally summed over the four SoCs on four cores of SoC 0. From $T = 256$ on, this last sum also spills into a DRAM-backed reduction on one core (0.84 ms). Every part scales with T; none scales with the number of routed rows.

Lane counts must divide 16. The compiler maps a stage onto a SoC's cores by its lanes per SoC: counts that divide 16 map evenly, while in our lane-split programs 17–19 lanes split the GEMMs unevenly and 20–32 lanes collapsed onto four cores per SoC (3.7–4.3 ms for a stage that runs in 0.5 ms on 16 cores).

Stages run one after another. The compiler schedules a layer's expert stages sequentially, in its own order; in our tiered programs a stage of L lanes per SoC ran one lane per core on L cores. Every additional stage adds a window of its own.

B.2 Expert-Parallel Compilation with an On-Device KV Cache

We compile with a weight-splitting degree of one across SoCs (`-mdts-mos=1`), which forbids cutting any weight matrix across SoCs. The lane axis is then the only way to distribute the batched expert GEMMs, and each SoC computes 16 whole experts per stage, one per core; the all-gather and re-shard of the down projection disappear, and the layer's cross-SoC traffic falls from 121.6 to 1.5 MiB.

The setting is global and also stops the compiler from splitting the attention projections by KV head. The cache update then comes out token-sliced while its input is whole, the compiler cannot pair the 96 cache inputs with their outputs, and all 192 KV buffers become host I/O. We restore the pairing at the graph level with *head-parallel attention*: the four KV-head groups of eight query heads each (Ainslie et al., 2023) become a leading batch axis of every attention operator, with per-group weights of shapes $[4, 2048, 1024]$ for queries, $[4, 2048, 128]$ for keys and values and $[4, 1024, 2048]$ for the output projection, whose four partial outputs are summed at the end. Under expert-parallel compilation the batch axis lands one group per SoC, the cache operations of KV head h stay on SoC h, and the program again exposes no KV buffer; the regrouped output projection changes only the FP16 summation order.

The group sum. The four partial outputs live on four SoCs. Our first version summed them with an einsum over the group axis, which the compiler ran on few cores: in a full-model profile of RankTier at $T = 512$ it took 2.39 of each layer's 7.73 ms (31%). Written as the elementwise tree $(G_0 + G_1) + (G_2 + G_3)$ per token tile, like the MoE layer's final sum, it spreads over all cores and saves 17, 67 and 130 ms per chunk in the naive program at $T = 128$, 256 and 512. This repair belongs to attention; the evaluation

keeps it as step $2'$ and uses it in every later program.

B.3 Lowering the Combine

Dense-path rewrites. Before the combine is replaced, three rewrites remove serialized work. (i) The first stage's outputs were added to rows gathered from the freshly zeroed accumulator; we drop this read of zeros. (ii) The per-SoC sum over lanes, one einsum that the compiler ran through DRAM on one core, is cut into 16 tiles of $T/16$ tokens. (iii) The routing's prefix sums over tokens, which give each assignment its row in the expert's compact input and ran as a serial `CumSum` on one core, become Hillis–Steele scans of $\log_2 T$ shift-and-add steps (Hillis & Steele, 1986). All three leave the output bit-identical. They repair this export and this compiler's lowering rather than change the combine's formulation, and the evaluation keeps them as steps of their own: 3a for the scans, 3b for (i) and (ii).

Ownership-preserving aggregation. The device gather does not zero-fill out-of-range indices, so a safe index and a mask are both required, and the per-SoC work must stay on the lane axis the compiler already splits over SoCs: written with global row indices or as explicit per-SoC slices, the same sum shipped 1539 MiB of expert outputs between SoCs per layer and made the full model 3.4–5.3× slower (§3.3).

Elementwise final sum. Written as 16 einsum tiles, the sum over the four partials ran on four cores per SoC; written as the elementwise tree $(P_0+P_1)+(P_2+P_3)$ per token tile, it spreads over all cores, and the compiler lowers it as a reduce-scatter that the next layer's input normalization completes.

B.4 Lowering the Rank Sort and the Tiers

One gathered constant. The counts come from the routing mask, and the per-SoC sort is a `TopK` over each SoC's 32 counts. Each tier's three weight matrices come from `Gather(bank, π)` on one constant that holds all 128 experts, the only gather form this compiler keeps fast, lowered to indirect DMA (§3.3). The constant is replicated, so every SoC holds the weights of all 128 experts, 22.5 GiB per SoC instead of 6.7 GiB, within the SoC's 32 GB; the algorithm itself needs only each SoC's own 32 experts.

Off the critical path. The routing mask, the prefix scans and the slot tables are computed once over all 128 experts in native order, independent of the sort; only a row permutation of these tables and the weight gathers wait for it. Sorting first and running the routing on permuted columns makes everything wait (12 layers: +2.5 ms over capacity-T padding, against +1.6 ms). Gathered weights are also sensitive to the compiler's maximum tile size: with the default, the first expert GEMM waits 0.085 ms for its first gathered tile, with 512 KiB tiles 0.032 ms. For the rank sort the best size is 512 KiB for $T \leq 256$ and 1024 KiB for $T = 512$ (Appendix E); other programs differ (Table 2).

Tier widths and count. Stages map evenly only when their lanes per SoC divide 16, run one after another, and use L cores for L lanes (Appendix B.1), so tiers take widths from $\{1, 2, 4, 8, 16\}$. At $T = 512$ the first tier is bound by the SoC's shared memory traffic: eight lanes of 512 rows finish in 0.8–1.0 ms, sixteen in 2.2 ms. We deploy three tiers of 8, 8 and 16 lanes (§4.3, §6.5).

B.5 Graph Transformations

RankTier is a set of ONNX graph transformations, about 1,200 lines of Python, applied to the Qwen3-30B-A3B graph that the vendor's QEfficient library exports for Cloud AI 100 (Qualcomm Innovation Center, 2024). Each step of the ladder (§6.1) is a separate pass, so every intermediate program can be compiled and measured. The passes locate operators by structure rather than by name (layer 0 numbers some nodes differently and folds a 64-lane constant into its stage), regenerate the T-dependent constants of the scans and tiles for each chunk length, and clone the stage subgraph once per tier with its own row order, weight gathers and capacity. A static `If` in the export that squeezes the token table is replaced by the equivalent `Squeeze` in every clone, and the aggregation's map from an expert to (tier, SoC, lane) becomes a set of small per-tier tables. The passes also build the evaluation's variants: the export's own combine, the aggregation formulations of Table 3, fixed expert order through the gathers, fixed-size blocks, and tier capacities read from input lengths for the overflow fallback.

Each chunk length is its own program, with sequence length T and context $2T$, MXFP6 matrix weights, FP16 activations, 16 cores per SoC, depth-first scheduling, expert-parallel slicing and retained state (Appendix A). Static programs occupy 25 GiB, programs with gathered weights 88 GiB. Every program returns each lane's token count in every layer.

C Routing Captures and Tier Calibration

Routing captures run the model's FP32 layers on the CPU and record every layer's top-8 experts. At $T = 128$ we use 50 chat prompts per workload; at 512 tokens, 8 GSM8K chunks and 4 each from HumanEval, MMLU and SWE-bench Lite; the 256-token calibration set holds 16 GSM8K chunks and both halves of each 512-token chunk (56 in total). The held-out GSM8K chunks of the evaluation use test questions 1000–1003 (256 tokens) and 1004–1011 (512 tokens), outside the calibration range. Table 6 gives the largest per-SoC load at selected ranks, the input to the tier

rule of §4.2.

Table 6. Largest load of the rank-r expert of a SoC over all calibration chunks, layers and SoCs.

T	$r=1$	2	4	8	9	16	17	32
128	127	122	105	33	32	14	12	3
256	256	223	134	58	50	25	25	4
512	508	424	240	101	89	51	50	10

A fixed layout calibrated per workload fares worse the more its traffic varies. With 25 calibration prompts of the same workload, 64–92% of that workload's held-out prompts overflow in some layer; calibrated on 175 prompts of the seven other workloads, 24–100% do. With 200 prompts from all eight, the fraction falls to 12–36%, and with random pooled subsets of 8, 32, 128 and 200 prompts to 100%, 70%, 27% and 21%. Under the per-SoC rank sort, ranks 17–32 need at most 12 rows at $T = 128$ in every layer of every workload.

D Additional One-Layer Measurements

On a one-layer replay (layer 2, MXFP6, expert-parallel), the combine and the capacities interact. With capacity T in both stages, a token-centric predecessor of token-owned aggregation saves 12%, 29% and 23% at $T = 128$, 256 and 512; fitted capacities save 6%, 14% and 13% with the dense combine but 9%, 15% and 25% once the combine is fixed, since the padded compute is then exposed. Both together are 20%, 40% and 42% faster than the naive layer.

E Tile Size of Gathered Weights

Table 7. Rank sort on 12-layer cuts, measured before the attention fix: latency change against the token-owned program at capacity T (default tiles) for three maximum tile sizes.

T	Default tiles	512 KiB	1024 KiB
128	+4.7%	**−1.4%**	–
256	−5.1%	**−18.2%**	−7.9%
512	−12.7%	−9.3%	**−15.9%**

Tile size. Table 7 shows the sensitivity of gathered weights to the compiler's maximum tile size. With default tiles the rank sort is 4.7% slower than the token-owned program at capacity T at $T = 128$; with 512 KiB tiles it is 1.4% faster. The best size moves with the chunk, 512 KiB up to 256 tokens and 1024 KiB at 512; static weights prefer the default only at $T = 128$ (small tiles slow them by 1–2%), while at 256 and 512 tokens 512 KiB tiles make the static program 8% and 16% faster (Table 2). Each program therefore uses its own best size. The tile size is the granularity of the compiler's row skipping (Appendix G); at $T = 128$ a lane is one tile at every size.

F Ladder Chart

Figure 5 plots Table 1 relative to the production compilation.

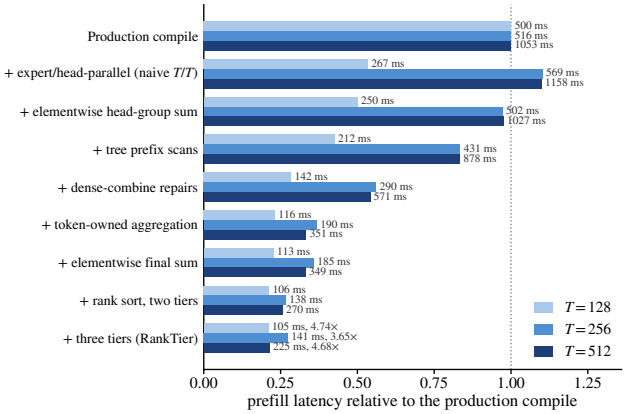


Figure 5. Prefill latency of the nine programs of the ladder relative to the production compilation at the same chunk length, from the joint fit, with the latency of each program and the speedup of RankTier.

G Profiles of the Attribution Programs

Table 8. Layer 1 of instrumented two-layer cuts at $T = 512$, before the attention fix, which does not change the MoE layer: MoE layer time, tensor-unit busy time summed over a SoC's cores (busiest SoC) and MXFP6 weight bytes read per SoC, per stage in execution order.

Program	MoE (ms)	HMX (ms)	Weights (MiB)
A, default tiles	6.87	26.0 / 21.1	53 / 56
A, 512 KiB	4.12	5.1 / 3.1	56 / 56
B, 1024 KiB	5.23	18.5 / 10.7	56 / 56
C, 1024 KiB	5.74	24.3 / 8.9	56 / 56
C, 512 KiB	6.45	4.2 / 1.8	56 / 56
D, 1024 KiB	5.07	27.3 / 0.6	56 / 49
E (RankTier)	3.65	0.8/3.3/0.35	49/28/28
Blocks, 64, dropless	12.6	31.4	337.5
Blocks, 64, trace-spec.	8.5	9.9	168.8

Tile predication. The export ends each expert lane with $\mathrm{Where}(r < n_e, y, 0)$, where n_e is the expert's run-time token count. The compiler lowers it as a range select that writes the zero rows directly and gates the work producing the others: it cuts the lane's rows into tiles, evaluates for each tile at run time whether it holds a row below n_e (a

compare and a reduction to a scalar), and runs the tile's activation gather, GEMMs and SiLU only if so. A lane with n tokens and row tile R therefore computes $R\lceil n/R \rceil$ rows, which holds exactly for every lane of every stage and SoC of the programs above: a 512-row lane is one tile at the default ($R = 512$), two at 1024 KiB and four at 512 KiB, and a lane spread over two cores is cut per core (RankTier's 128-row lanes: $R = 64$). Per SoC, A computes 13.8–15.4k of its 16.4k rows at the default and 3.5–3.8k at 512 KiB, B and C 6.9–7.7k at 1024 KiB, and RankTier 3.3–3.5k of 6.1k. An expert's weight load and dequantization feed all of its lane's tiles: for an empty expert they are skipped only when the lane is one tile (A at the default, the 64-row tiers of D and RankTier) and run otherwise. At 512 KiB the gathered program C computes 128-row tiles like A but spills the activations of its GEMMs to DRAM (95–97 MiB written and read back per SoC, against none for A), which lengthens its stages. Both rules are read off the traces; the compiler does not document them.

H Aggregation Phases: Lane-Indexed against Step 5

Table 9. Aggregation of layer 1 of instrumented two-layer cuts at $T = 512$ (one sample), by phase: operations, busy core-ms summed over the 64 cores, and MiB multicast between cores. Last row: from the last expert GEMM to the end of the final sum, per SoC.

	Step 5			Lane-indexed		
Phase	ops	core-ms	MiB	ops	core-ms	MiB
Index build	6913	27.5	0.5	1040	3.3	0.4
Gather	762	66.7	0	4430	15.0	0
Mask multiply	128	1.4	0	3072	4.6	0
Stage add	64	0.7	0	1024	1.6	0
Sum to partials	304	2.4	1.9	2288	47.1	241.9
Final sum	1248	0.7	1.9	1250	0.9	1.9
After last GEMM	1.22–1.69 ms			3.10–3.68 ms		

Table 9 classifies the aggregation's operations by phase. Both formulations gather about the same bytes (898 and 924 MiB) and keep them on their SoC: cross-SoC traffic in the MoE layer is 14.0 and 12.1 MiB. They differ in the sum to the per-SoC partials. The lane-indexed formulation sums each token tile over a SoC's 16 lanes, which sit on 16 different cores, so every tile's sum pulls its operands from all cores; its 16 tiles span 3.0–3.6 ms per SoC, and their windows add up to 4.6–5.6 ms with only 1.7–2.0 ms of overlap. Step 5 sums each token's k rows on the core that owns the token. Its index build is heavier, but it runs while the expert stages do. In the profile, the window from the last expert GEMM to the end of the final sum is about 1.9 ms longer per layer for the lane-indexed formulation; the uninstrumented full models differ by 1.1 ms per layer (401.7 against 349.2 ms over 48 layers), so the instrumentation overstates the gap. The profiles' critical-path tables hold no usable rows, so these windows locate the difference without decomposing it along a dependency chain.

I The TokenCombine Operator

Interface. TokenCombine is a custom operator written against the SDK's C++ interface. Its inputs are a stage's expert outputs in the layout of §4.5, $[4, 16C, d]$ in FP16, and per-SoC row indices $[4, Tk]$ that hold -1 for a token's experts on other SoCs; its output is the per-SoC partials $[4, T, d]$. It thus replaces step 5's gather, mask and sum over k, while the graph keeps the index construction, the add of the two stages' partials and the elementwise sum across SoCs. Its configuration declares the output tileable along all three axes; for an output tile $[b_0{:}b_1, t_0{:}t_1, d_0{:}d_1]$ the operator requests the data tile $[b_0{:}b_1, 0{:}16C, d_0{:}d_1]$ and the index tile $[b_0{:}b_1, t_0k{:}t_1k]$, so a tile on SoC c reads only c's expert outputs. The kernel converts each indexed row from FP16 to FP32 with HVX vector instructions, accumulates a token's rows in FP32 and converts the sum back to FP16 once; it asks for its operands in the core's vector memory.

Integration. In a four-SoC test of one stage at $T = 128$, the operator's output is bit-identical to step 5's aggregation. In the full model at $T = 512$ its tiles land on all 64 cores, and the cross-SoC traffic of the MoE layer stays at 14.1 MiB. A scalar version was 6.5–7.8× slower than step 5 on two layers, compute-bound at about 26 ms of busy time per core and layer; the HVX version runs the two layers in 14.8 against 15.7 ms. The compiler did not place the data input in vector memory and warns about it 26,112 times, without effect on the result. It instantiates the kernel at every output tile and hands each SoC's code to one single-threaded `hexagon-clang++ -O3` run of about 2.7 h, so a full-model program takes 650–656 minutes to compile.

Numerics. The last-position logits differ from step 5's by 1.68×10^{-2} relative L_2, with a largest absolute difference of 0.273 for logits up to 24.7, the same top five and the same next token; both programs repeat bit-identically, so the difference is deterministic. The kernel rounds each stage's FP32 partial to FP16 once. Step 5 adds the two stages slot by slot, which is exact because each slot is non-zero in one stage only, and then reduces the eight slots with the compiler's `ReduceSum`, whose accumulation precision is undocumented. Formulations that reorder the same sums differ from step 5 by 0.7–3.3×10^{-2}; those that keep its order are bit-identical. One matching next token is not evidence of general equivalence.

J Block Scheduling by Block Size

Table 10. Block scheduling against RankTier on two-layer cuts at $T = 512$, measured before the attention fix (ms; RankTier 15.9). Blocks per SoC in parentheses.

B	dropless	calibrated	trace-specialized
16	38.3 (288)	28.0 (176)	19.7 (96)
32	30.3 (160)	21.2 (96)	18.1 (64)
64	24.6 (96)	20.1 (64)	**17.9** (48)
128	**24.0** (64)	= dropless	18.1 (32)
vs. RankTier	1.51×	1.26×	1.12×

K Rejected Alternatives

Table 11 lists the alternatives we rejected. Splitting a loaded expert over two lanes halves the first tier's GEMM time in isolation, but the compiler keeps the 16 lanes on eight cores per SoC and dequantizes each expert twice, so the program gets 6% slower. Co-locating experts that fire together cuts the SoCs a token touches from 3.63 to 2.42, but it doubles the order statistics the lower tiers must cover (at $T = 512$ from 50 to 85 rows at rank 17) and targets an exchange that is only 3% of the model.

Table 11. Alternatives we built or analyzed and rejected ($T = 512$ unless noted).

Alternative	Result	Reason
Sort first, then route on permuted columns	+2.5 vs. +1.6 ms (12 layers, T=128)	all routing waits for the sort
Row-chunked expert GEMMs	+25%, +66% (one layer)	each chunk re-streams and re-dequantizes the weights
One partition per SoC	1.7× slower	the runtime runs partitions in sequence
Per-layer first-tier capacity	−10–15% of its rows	held-out chunks overflow in 1–5 layers
Co-locate experts that fire together	3.63→2.42 SoCs per token	doubles lower-tier loads; the exchange is 3%
Split first-tier experts over two lanes	+6% (two layers)	16 lanes stay on 8 cores; 1.8× dequantization
Four tiers	−8.6% vs. −14.1%	one more sequential window
No depth-first scheduling	+1–2%	stages still run in sequence

L Stage Windows

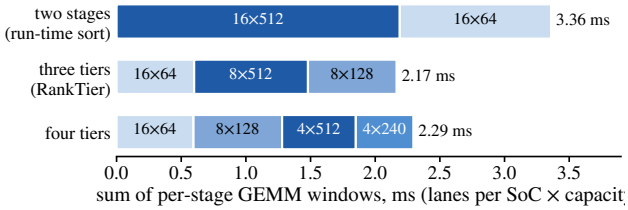


Figure 6. GEMM window of each expert stage in execution order at $T = 512$, two-layer profiles before the attention fix; segments are the per-stage medians over SoCs, labeled with lanes per SoC × capacity.

M Accuracy per Program

Table 12. Relative L_2 error of the last-position logits against FP32, and the next token (✓: matches FP32). Programs in one row give identical logits.

Program	$T = 128$	$T = 256$	$T = 512$
1 Production	0.188 ✓	0.089 ×	0.095 ✓
2 Naive T/T	0.172 ✓	0.077 ✓	0.094 ✓
2′–3b Head sum, rewrites	0.199 ✓	0.072 ✓	0.087 ✓
4 Token-owned	0.156 ✓	0.082 ×	0.089 ✓
5 Elementwise sum	0.163 ✓	0.087 ×	0.092 ✓
6–7, A–E Rank sort, RankTier	0.188 ✓	0.091 ×	0.096 ✓
TokenCombine	–	–	0.095 ✓